Making Uber’s Experiment Evaluation Engine 100x Faster

uber.com/en-PL/blog/making-ubers-experiment-evaluation-engine-100x-faster

Akshay Jetli

October 3, 2024

Stay up to date with the latest from Uber Engineering

Follow us on LinkedIn

Introduction

This blog post describes how we made efficiency improvements to Uber's Experimentation platform to reduce the latencies of experiment evaluations by a factor of 100x (milliseconds to microseconds). We accomplished this by going from a remote evaluation architecture (client to server RPC requests) to a local evaluation architecture (client-side computation). Some of the terminology in this blog post (e.g., parameters, experiments, etc.) is referenced from our previous blog post on Uber Experimentation. To learn more, check out Supercharging A/B Testing at Uber.

Experimentation is used heavily throughout Uber's backend microservices, mobile apps, and web surfaces to measure improvements of product launches across all business units– Delivery, Mobility, Freight, Uber For Business (U4B), etc. We will cover the current and new architecture of the system as well as technical challenges, and learnings. This blog post specifically focuses on latency improvements to **backend** Golang microservices rather than mobile or web-based experimentation. We are also focusing on A/B tests rather than other experimentation designs.

Background

In 2020, Uber began a long journey to rewrite the A/B testing platform (internally referred to as Project Citrus). One of the key architectural decisions for this new system was to unify Uber's existing backend configuration platform, Flipr, with the Experimentation platform. We built Experimentation to exist as a temporary override layer on top of Flipr configurations (also referred to as parameters)in order to perform randomization, conduct causal inference, and help teams determine the best performing value to use for their parameters.

Below is an example to illustrate the difference between a parameter and an experiment. Here we have a **parameter**, which controls the button color of a specific screen in the Rider app. The default value is red, but in some situations (e.g., if the user is in *city_id = 1*) the value would be blue for *all users* in that city.

```
parameter = {
    "name": "button_color",
    "default": "red",
    "exceptions": [
        {
            "test": "city_id: 1",
            "value": "blue"
        },
    ]
}
```

Figure 1: Parameter representation.

These parameter rules engines are evaluated using a Flipr client:

Figure 2: Flipr client parameter evaluation.

The calls to the **get** method return different parameter values based on the runtime arguments passed in (e.g., *city_id*, *user_id*, *app_version*, etc.).

An **experiment** allows us to randomize the behavior of the parameter for some unit of measurement (e.g., users, sessions, devices, or trips). While an experiment is active, the parameter has additional **parameter metadata**, noting down the associated experiment that would override the behavior of the parameter. When the experiment is concluded, this metadata is removed.

```
parameter_metadata = {
    "experiment_override": "button_color_experiment",
}
```

Figure 3: Parameter metadata representation.

In the below example, 2 different users in *city_id = 1* could get 2 different values of a button color based on which **treatment group** they belong to in the experiment (the below A/B test is set up as 50% treatment and 50% control and is randomized by *user_id*).

```
experiment = {
    "name": "button_color_experiment",
    "default": "red",
    "exception": [
        {
            "test": "city_id: 1 AND treatment_group: control",
            "value": "blue",
        },
        {
            "test": "city_id: 1 AND treatment_group: treatment",
            "value": "green"
        }
    ],
}

experiment_metadata = {
    "treatment_group_distribution": {
        "control":   [0, 49],
        "treatment": [50, 99],
    },
    "randomization_unit": "user_id",
}
```

Figure 4: Experiment and experiment metadata representations.

The **experiment** and **experiment metadata** are read by the Flipr client and the return value of any parameters impacted by the experiment will vary based on the experiment randomization (e.g., *user_id = 123* may get blue because they are part of the control group, while *user_id = 456* would get the value green):

```
value = flipr_client.get("button_color", { "city_id": 1, "user_id": 123})   # "blue"
value = flipr_client.get("button_color", { "city_id": 1, "user_id": 456})   # "green"
```

Figure 5: Flipr client parameter evaluation with experiment override.

We log the exposure of users to experiments so that we can measure the impact that these changes have on key business outcomes, such as completed trips or gross bookings.

Problem Statement

Historically, all experiment evaluations happened through network calls to a centralized microservice, whereas evaluation of parameters with no active experiments happened locally (referred to as “local evaluation”). As part of project Citrus, we kept a pattern where all backend microservices made RPCs to a central service to evaluate active experiments. This approach allowed us to ship the new platform relatively quickly and it reduced migration cost for our customers to adopt the new platform. All of our clients (backend systems, mobile devices, and web services) adopted the new platform by making RPC calls to the new evaluation service (internally referred to as **Parameter Service**).

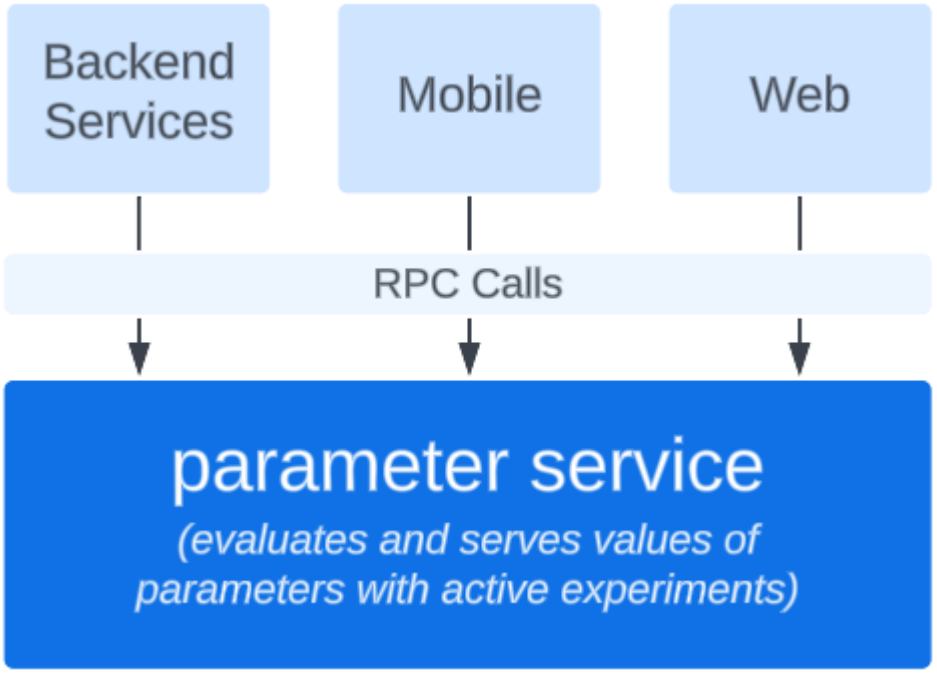

Figure 6: Parameter Service Architecture (RPC evaluation).

However, this pattern also left several problems unaddressed for backend callers of experimentation, that we wanted to solve as part of our experimentation platform rewrite.

- **Latency**: RPC based evaluations are relatively slow (p99 latency of ~10 ms). Given the real-time nature of the Uber marketplace, there are many applications that needed their experiments to be evaluated faster in order to onboard to the new platform.
- **Reliability**: The parameter service became a single point of failure. To mitigate this concern, we invested heavily in the reliability of parameter service, eventually maintaining an availability SLA of 99.95% for each of its endpoints. We also encouraged teams to rely on a fallback value in case of RPC failure. However, given the scale of the system and the fact that hundreds of teams and microservices call parameter service to evaluate thousands of parameters and experiments at any given time, the behavior of every fallback could not be guaranteed. Occasionally, the fallback value would become unsafe, resulting in outages that would then need to be promptly fixed.
- **Developer Productivity**: In order to reduce the number of requests to parameter service, we introduced a prefetch mechanism. As part of this mechanism, a backend service would send a batch RPC request to parameter service to evaluate and return **all** of the parameters under active experimentation needed to process business logic for that specific request. The experiment values and exposure logs returned from the parameter service would be stored in the local in-memory cache of the backend service.
- The backend app would then **get** the value of each individual parameter needed in that request (which was impacted by an experiment) exactly when it was needed, and this is the point at which we would emit an exposure log for the user entering the experiment.

While this batching helped reduce the total number of RPC calls, it also meant that backend developers needed to be cognizant of the prefetch mechanism while coding their application. They would need to plan for which parameters to prefetch upfront so that these parameter values would be available later. A failure to do so would result in unexpected bugs that would then need to be debugged and fixed, resulting in inefficiency.

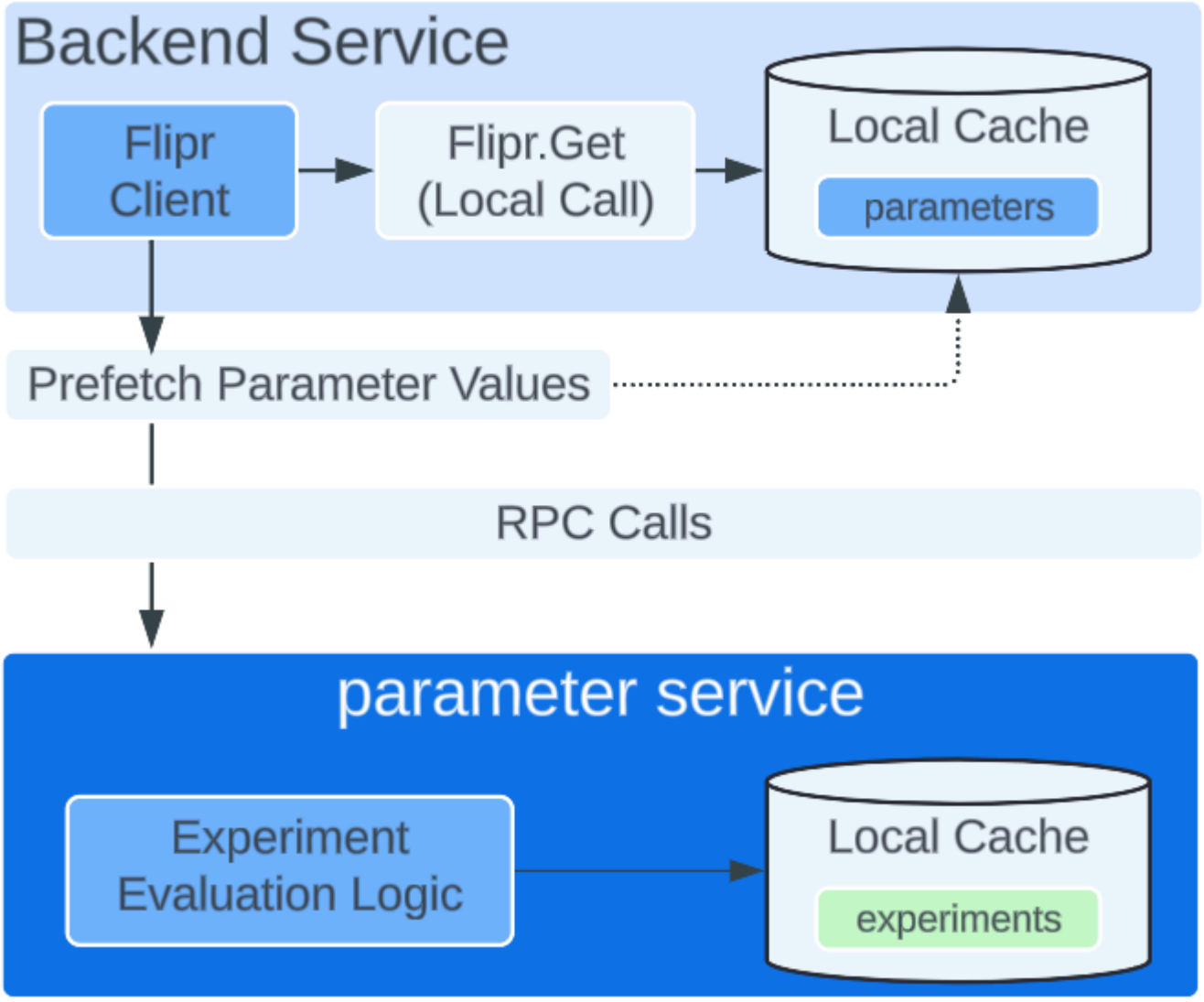

Figure 7: Initial Architecture: Prefetch parameters mechanism.

We believed that all of these concerns could be addressed by a new architecture involving client-side evaluation of experiments (and parameters) without any RPC requests to the parameter service.

Architecture

Uber's existing architecture involved a distribution mechanism where Flipr parameters were sent to all host agents at Uber (see previous eng blog for more details). Each microservice running on such host machines would rely on the Flipr client to periodically bootstrap these parameter rules engines by reading from file caches which results in a much higher availability than online RPC architecture.

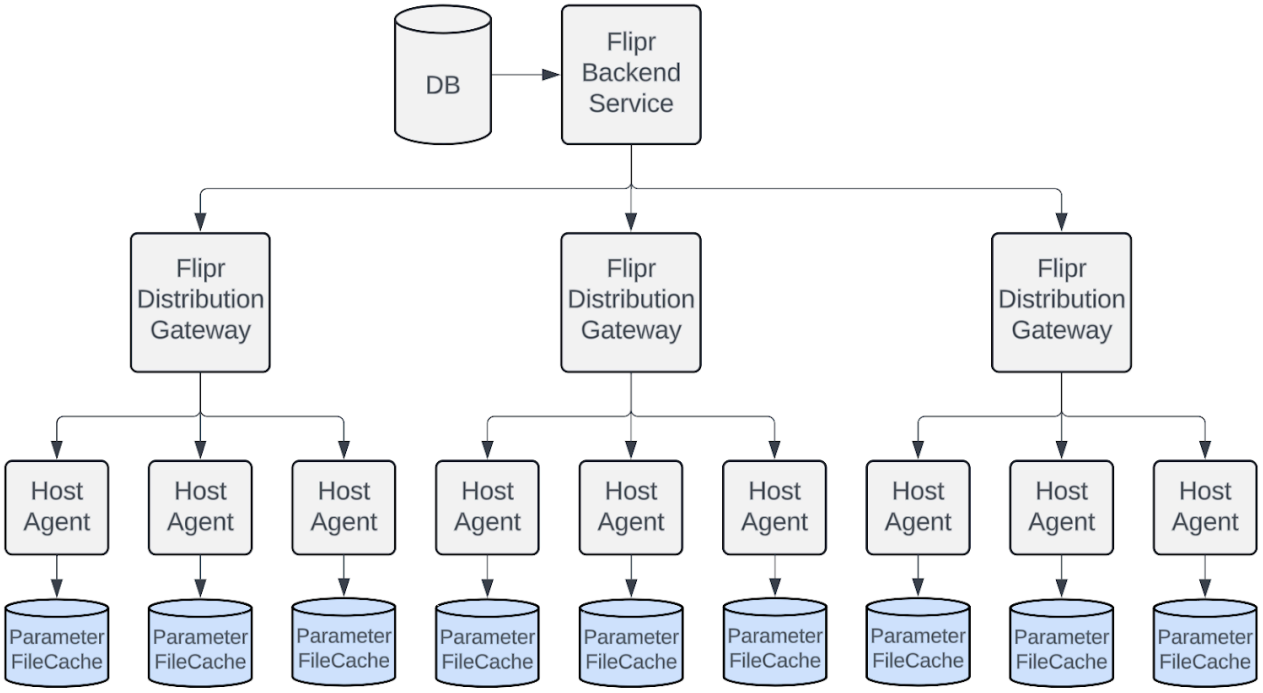

Figure 8: Parameter host agent distribution.

One of the major changes in the new architecture was to distribute *experiments* to all host agents by using the same distribution layer as regular Flipr parameters. This change in the distribution layer would unlock the ability for each microservice to locally read experiment data without any RPC to Parameter Service at all.

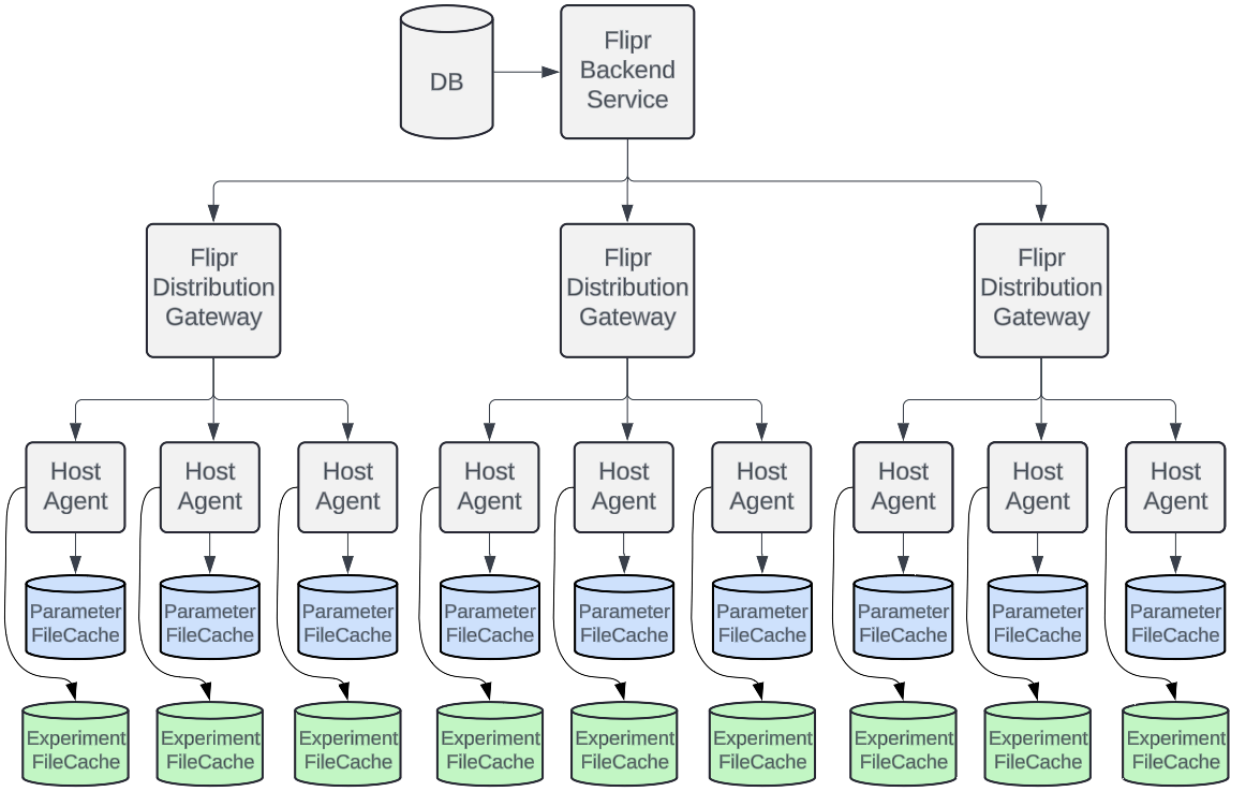

Figure 9: Experiment host agent distribution.

The next step for us was to port all experimentation specific business logic of the Parameter Service into an **ExperimentPlugin**,which would be embedded directly within the Flipr client. We intended to pilot this new capability directly within the Parameter Service before exposing this new capability of Local Experiment Evaluation to external microservices. The benefit of being the first users of Local Experiment Evaluation was to allow for **rigorous shadow testing** to ensure correctness and performance parity.

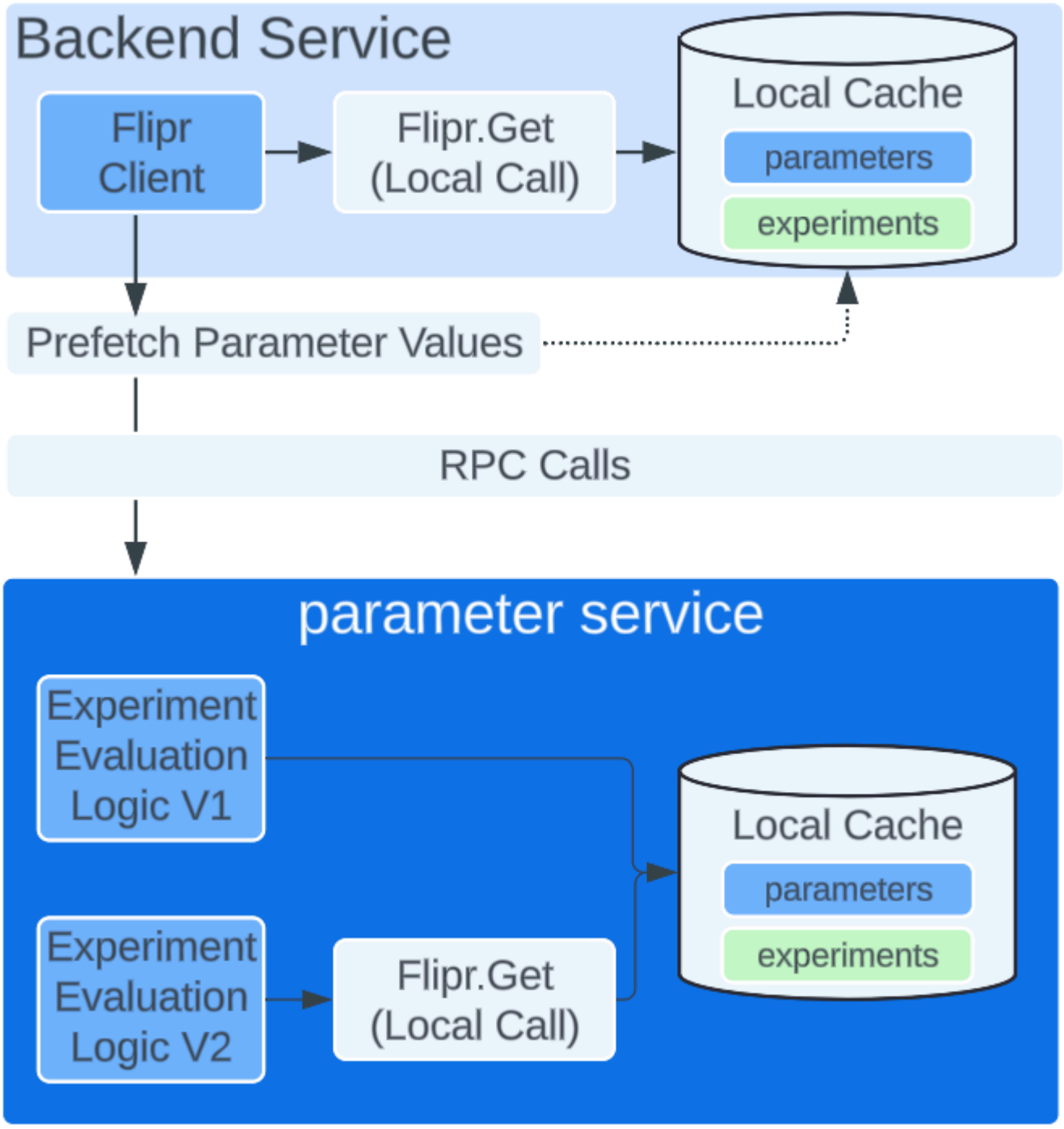

Figure 10: Intermediate Architecture: Shadow evaluations in Parameter Service.

After a thorough shadow testing process through which we uncovered and addressed 13 separate bugs, the final step in this journey was to allow teams to directly opt in to use the Local Experiment Evaluation capabilities and remove their usage of the PrefetchParameters API entirely.

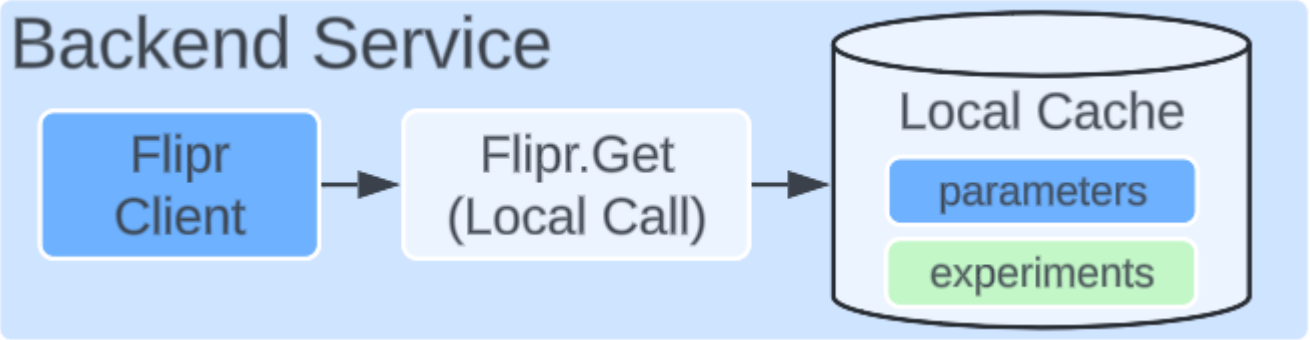

Figure 11: Final Architecture: Direct usage of Local Experiment Evaluation in backend microservices.

Challenges & Learnings

1) Verification at scale

One of the major challenges of building this new local evaluation functionality was to ensure that it was performing correctly at scale. In order to do this, we implemented a shadow testing capability to ensure that a sampling of requests that went to the V1 legacy evaluation code path would also trigger a separate async goroutine to evaluate and compare results with the V2 local experiment evaluation code path.

The scale of such evaluations was enormous (roughly 20 million evaluations per second) and required heavy sampling to prevent any degradation to Parameter Service to perform these comparisons. Additionally, the types of evaluations varied greatly from backend, mobile, and web surfaces. For this reason, we found it necessary to prove that every mismatch we encountered was explainable. And eventually we achieved a match rate >99.999% (1 explainable discrepancy out of 100,000 evaluations on average).

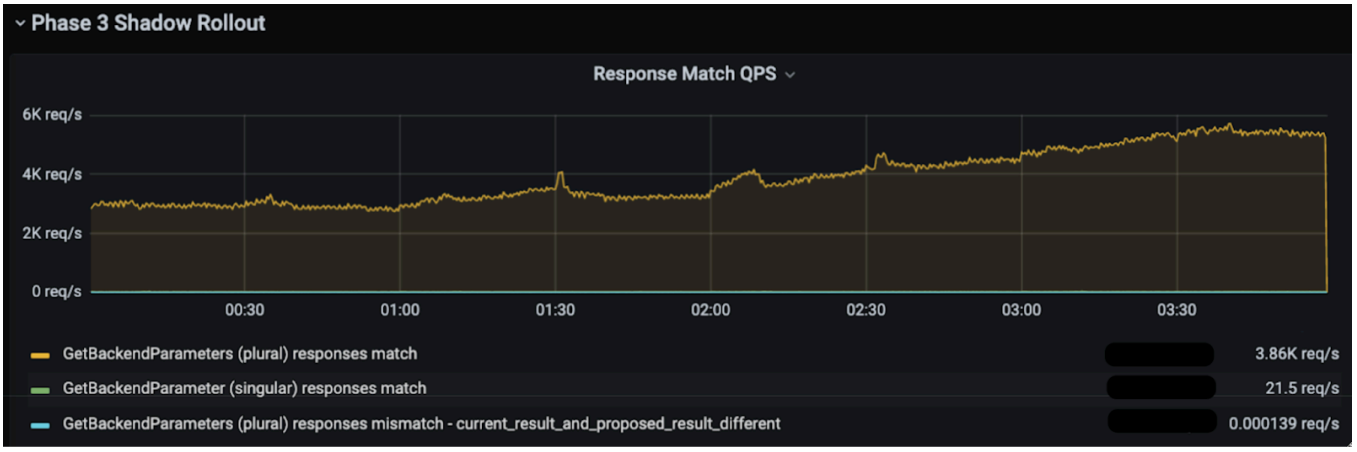

Figure 12: Shadow evaluation match rate.

Some of the key learnings from this process included:

- **Known discrepancies during parameter or experiment updates** – Whenever users updated the values or rules engine for parameters or experiments, this would predictably result in a race condition. The V1 evaluation methodology might use version X of the parameter or experiment whereas the shadow comparison using V2 methodology could use version X+1 based on the timing of Flipr distribution. This became the *only* source of acceptable discrepancies in the system.
- **Bugs in the old methodology** – Although 100% parity with the old system was the goal, at times we decided to "fix forward" and address minor bugs and edge cases in the legacy system instead of the new system. This proved to be a major benefit of this migration: in addition to providing new functionality, we were able to fix difficult-to-detect edge cases in the legacy system too.
- **Race conditions and timeouts** – Similar to the issue of "known discrepancies" during parameter or experiment updates, we realized that some subtle race conditions could affect our shadow evaluation match rate. One example was to set an adequate timeout for the shadow evaluation–it should mimic the same context deadline provided to the original evaluation to do an apples-to-apples comparison not just for logical correctness, but also performance.

2) Logging volume concerns

The dramatic improvement in evaluation speed introduced a risk to experiment log production and processing. Evaluating a parameter that is being experimented on may produce an exposure log to Apache Kafka®. If parameters can be evaluated 100 times faster, this means logs can potentially be produced 100 times faster too, and there is a risk of potentially overwhelming the infrastructure that produces, distributes, and processes the logs. In particular, we were concerned about dropping experiment logs due to hitting rate limits on our Kafka topic (either byte rate or message rate throughput limits).

One common experimentation use-case where a large cohort of users can be processed in bulk is producing marketing copy for an email campaign involving hundreds of millions of users.

To mitigate this risk we did the following:

- **Telemetry & alerting on log volume production** – First of all, we added telemetry to understand how many experiment logs were produced per second by each client, and alerting our on-call team if any one of them was producing greater than 5% of the total quota for our Kafka topic. This allowed us to identify hotspots and build solutions to address them.
- **In-memory LRU cache** – Because our experiment log processing pipeline only cares about the first time each user enters the experiment, we were able to deduplicate an enormous volume of logs. We decided to use a highly performant in-memory LRU cache (golang-lru) to reduce redundant logs, and ultimately were able to deduplicate roughly 80% of logs.

- **Parameters within the Flipr client library** – Having the ability to add “internal” parameters used within the Flipr client library and ExperimentPlugin became crucial to developing features safely. In order to avoid circular dependencies, we had a separate mechanism to load these “internal parameters” at file cache bootstrap time vs. providing runtime context to evaluate them. In case of emergency, we were able to entirely disable local evaluation using these internal Flipr client parameters without having to re-deploy services.

3) Decentralized evaluation

Lastly, a new challenge that we had to face was the fact that experiment evaluations were now decentralized as part of a client library versus a single centralized microservice. One of the major benefits of RPC-based evaluation, especially in the initial development phases of project Citrus, was that any issues with the core evaluation logic could be easily reverted by redeploying a single microservice. We would encourage others to only move towards client side evaluation once the platform is in a mature state, because reverting multiple separate microservices involving separate versions of client libraries is a much more costly and time consuming exercise.

Some of the tools which helped us with this paradigm change include:

Client library version tracking – Even though our backend microservices are deployed at a regular cadence, it isn’t always the case that every single microservice at Uber is up-to-date with the latest client code changes. Client version tracking telemetry allowed us to more easily identify services which were using outdated or buggy versions of the SDK.

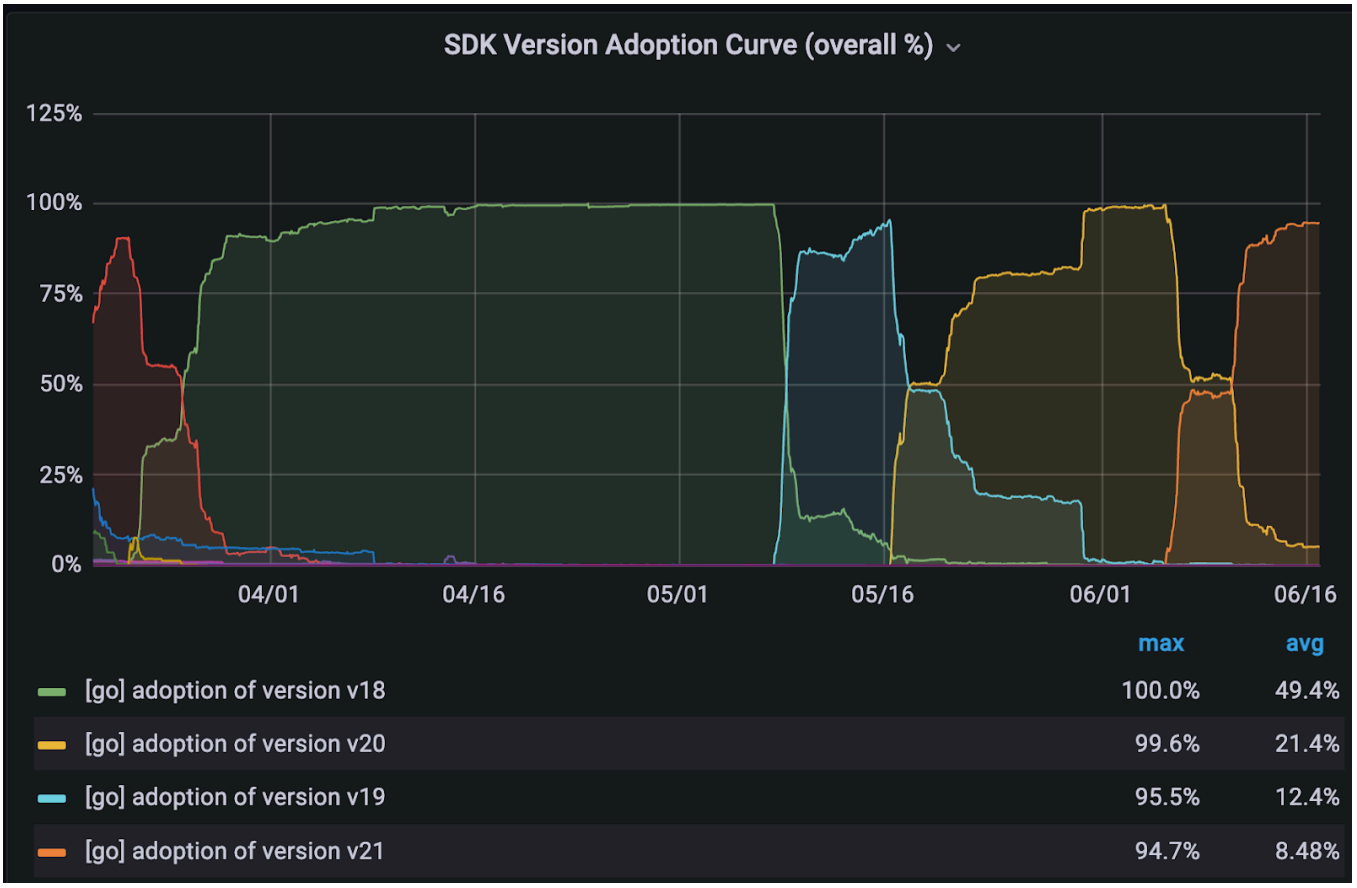

Figure 13: Client library version adoption curve

Bulk deployment infrastructure – Up is the platform at Uber which manages orchestration and code deployment of thousands of microservices across millions of host instances. We were able to add capabilities to bulk deploy services in a worst-case scenario (e.g., broken client version released).

Impact

Ultimately, the new architecture proved to be a success for many use cases at Uber. The p99 latencies of experiment evaluations dropped by a factor of 100x (from 10 ms to 100 µs).

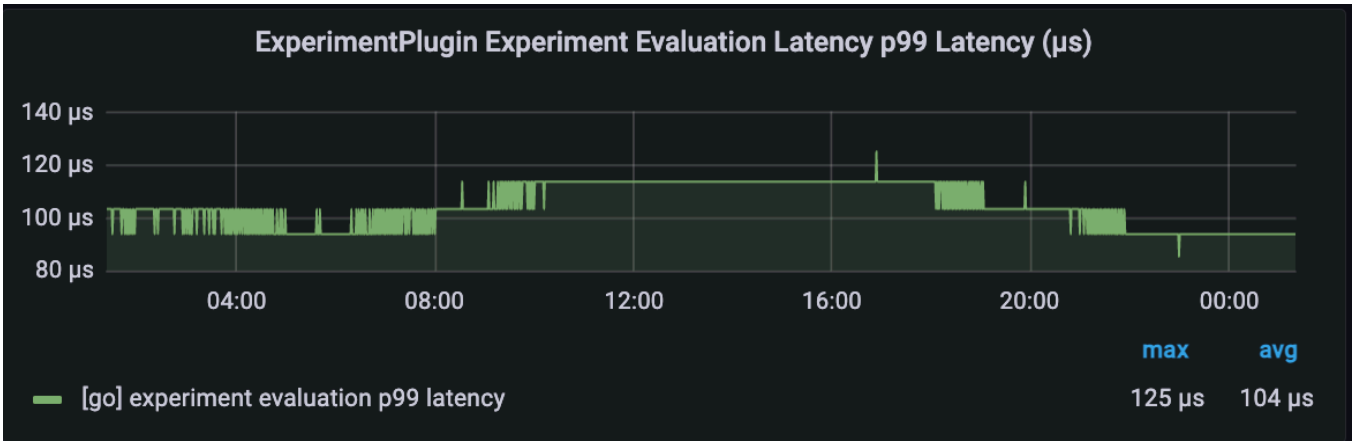

Figure 14: Experiment Evaluation p99 latency.

After the initial release of this feature in H2 of 2023, we strategically targeted the rollout to the Go microservices contributing the most to experiment evaluations, resulting in over 100 services and nearly 70% of experiment traffic so far.

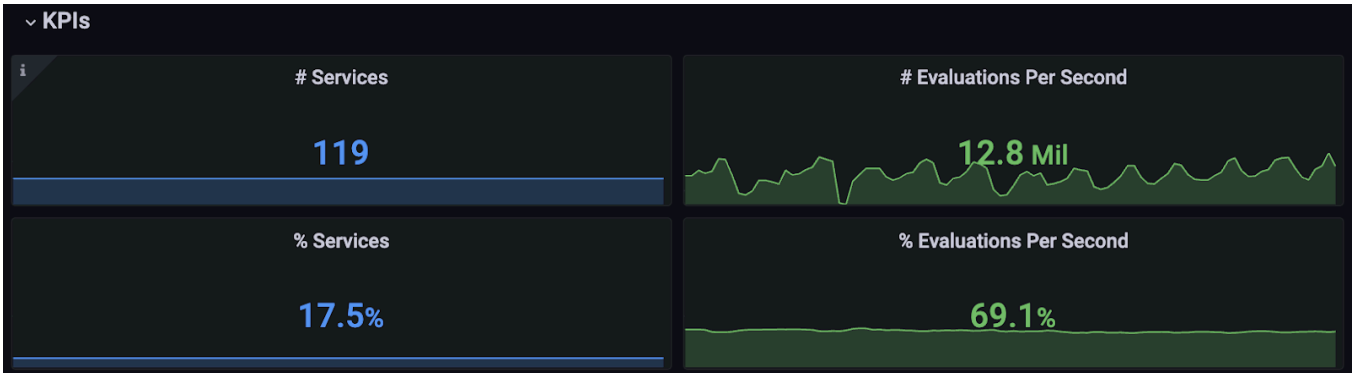

Figure 15: Local Experiment Evaluation adoption KPIs.

Additionally, we have been able to observe some noticeable improvements in user-facing functionality in Uber's applications. For example, the end-to-end backend latency of the UberEats search suggestion indexing reduced by 20% (p99 latency went from 250 ms to 200 ms), which translated to an appreciable improvement in the mobile app experience.

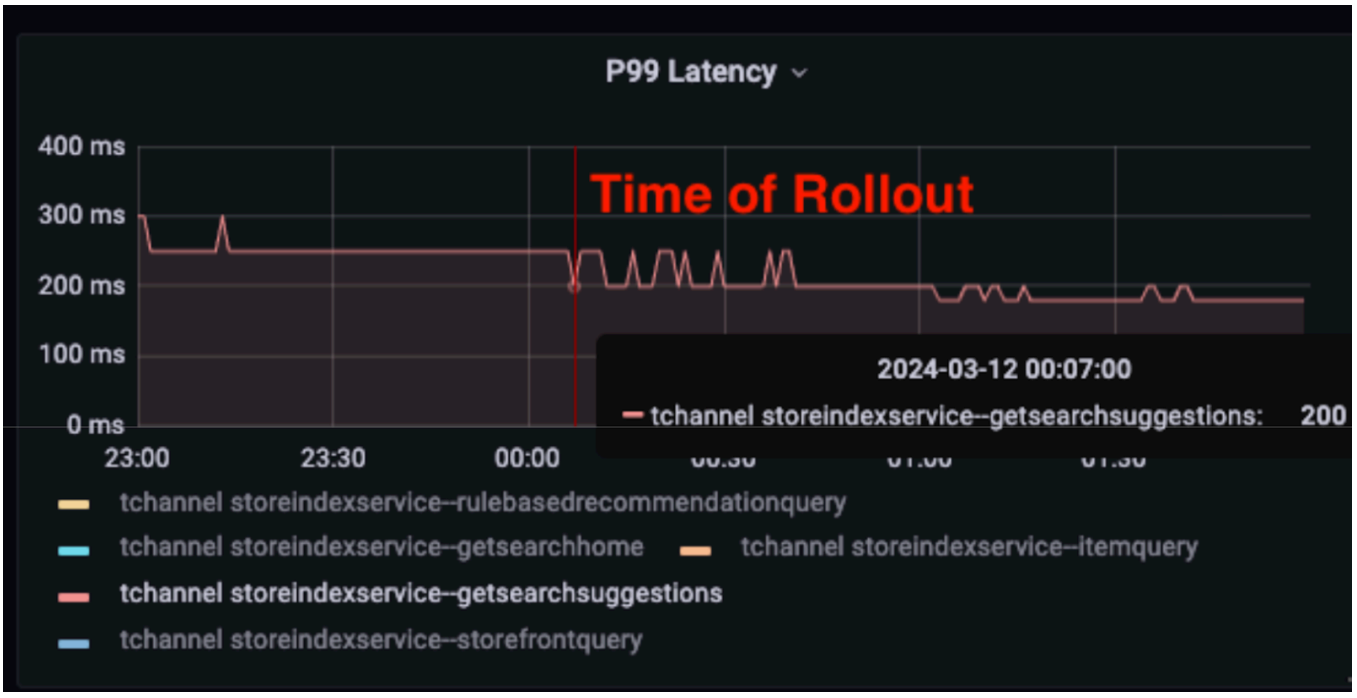

Figure 16: Impact of Local Experiment Evaluation for UberEats store index service.

Next Steps

This first iteration of Local Experiment Evaluation was for services using Golang, which serves the majority of our backend experimentation. The immediate next step is to also include Java, which is the other main backend service language that is supported at Uber.

Beyond this, there are plans and ongoing work to move the complexity from the clients to the server side. Complex clients are harder to rollout than server-side changes, and having complex clients in two or more languages adds duplication of effort and risks of divergence.

A more expressive language would allow us to move some of this complexity to the server. Instead of the clients parsing the experiment rules engines on the client using an ExperimentPlugin pattern for both Golang & Java, the system could parse this logic on the server, and output simpler programs in the more expressive client language to be executed by both Golang and Java clients. This would lead to thinner clients, and more server side control. The details of this will be the subject of future blog posts.

Conclusion

In summary, we replaced the RPC-based experimentation system with a new feature that evaluates the experiment rules engines locally by reading from file caches that are distributed to all host agents. Adding local evaluation for the experimentation stack helped improve the following:

- **Latency:** Evaluation latency dropped by a factor of 100x (p99 10 ms to p99 100 μs), unlocking new integration opportunities
- **Reliability:** We removed a single point of failure on a single high-traffic microservice, replacing it with a more reliable distributed configuration pipeline
- **Developer Productivity:** Developers no longer need to worry about prefetching parameters in a batch request and caching them to avoid the RPC overhead, making code less complex

We learned many lessons from this initiative, which may be useful for others attempting to build an in-house Experimentation platform or make large scale updates to their existing platform:

- **Start with Centralized** – Creating a new experimentation platform which returns evaluation results through RPC calls to a centralized microservice is advisable, especially during the initial period of rapid new feature development
- **Decentralized when mature –** When the platform is mature, switching to a decentralized library architecture unlocks many benefits (as listed above)
- **Safe development of libraries** – To make (decentralized) library development safer, utilizing concepts like lightweight feature flagging within the client library, having strong telemetry and alerting, and tracking library version usage across microservices is important
- **Faster evaluations can mean faster log emission** – Making sure that the experimentation logging infrastructure can handle increased volume is critical when moving towards a client side, low latency evaluation architecture

Acknowledgments

This project would not have been possible without the contributions of numerous people on the Experimentation team, Flipr team, and pilot customer teams (Rider Experience, Ads Delivery Platform, and Earner Incentives) who helped us release Local Experiment Evaluation for Go services!

Apache®, Apache Kafka, Kafka, and the corresponding logo are either registered trademarks or trademarks of the Apache Software Foundation in the United States and/or other countries. No endorsement by The Apache Software Foundation is implied by the use of these marks.
Cover image by Iffany via Pixabay.
Akshay was formerly a Sr. Staff Engineer at Uber working on the Experimentation platform and focused on building highly performant, scalable backend microservices and libraries. He is now a Senior Manager leading the Experimentation Engineering team.

Deepak is a Principal Engineer at Uber. He currently works on Uber's B2B products and platforms with a focus on re-architecting several core entities and systems to unlock both engineering and business efficiency. Previously, as a senior IC of the experimentation platform at Uber, he worked on several aspects of the platform that are used by every team in Uber to conduct effective and accurate A/B experiments. Deepak has expertise building scalable, reliable software systems across a wide array of domains such as Cloud-Agnostic Machine Learning infrastructure, Cloud File storage, Configuration Systems, Virtualization, and Disaster Recovery. He earned his PhD in Computer Science from Purdue University, where his doctoral thesis title was 'Highly Available Storage Systems'.

Sergey is a Senior Manager leading the Experimentation Science team. As part of this role he has led the re-architecture and rebuild of Uber's experimentation platform, and prior to experimentation Sergey worked on subscriptions products at Uber.

Andy is a Sr. Staff Engineer at Uber working on Flipr, configuration systems and challenges related to keeping Uber's systems reliable, scalable and productive.

Posted by Akshay Jetli, Deepak Bobbarjung, Sergey Gitlin, Andy Maule

Category:
Engineering

Backend